

A Cross-Adapter Benchmark of Differentiable Rigid-Body Simulators

Shubham Singh
shubham.singh091@gmail.com

May 2026

Abstract

Differentiable rigid-body simulators promise gradient-based controllers that close the sample-efficiency gap with classical robotics, but the field has produced a dozen credible options in three years with no calibrated comparison. We benchmark eight 2024–2026-vintage simulators (MJX, Brax, Drake, Pinocchio, JaxSim, Newton, Genesis, TDS) through a unified **SimulatorAdapter** contract whose numpy-only boundary lets one Docker image hold every adapter simultaneously. Five experiments cover gradient correctness against analytical and FD references, forward+gradient wall-clock latency, scaling to 48 DoF, gradient-based learning vs. a CEM baseline at $n=5$ seeds with 95 % bootstrap CIs, and adapter usability. Findings: (i) MJX’s gradient on the Franka Panda MJCF is wrong independent of the GPU XLA compile pathology – cosine -0.13 vs FD when the compile does finish, and on a 16-GB host the CPU compile runs out of memory before finishing; (ii) Brax’s `kinematics.forward` assumes one joint per non-world body and fails on the menagerie Panda’s fixed-link-to-world structure (deprecated upstream, not our fix); (iii) two “differentiable” simulators (Genesis, TDS) expose no usable Python autodiff path; (iv) Newton’s float32 forward dynamics needs an adapter-declared FD step hint to pass gradient correctness on Panda; and (v) Pinocchio’s analytical Featherstone Jacobian – through a tangent-space **QuadraticCost** and `pin.dIntegrate` chain rule – closes a working SHAC loop on a floating-base Go2 (9×10^4 cost reduction in 120 iterations on a smooth in-air task), while MJX’s differentiable-contact step closes the same SHAC loop on Go2 with floor contact ($50\times$ cost reduction). All experiments are reproducible from a single `./scripts/reproduce.sh` with byte-identical JSON output gated by `SOURCE_DATE_EPOCH`.

1 Introduction

Differentiable simulators promise to close the gap between gradient-free reinforcement learning and the substantially more sample-efficient gradient-based controllers used in classical robotics. Eight simulators in active 2024–2026 development claim some form of differentiability: MuJoCo XLA (MJX), Brax, Drake (via AutoDiffXd), Pinocchio (Featherstone analytical Jacobians), JaxSim, Newton (warp.sim’s successor, built on NVIDIA Warp), Genesis (Taichi AD), and TDS (TinyDiffSim). Each makes different choices about: AD direction (forward vs reverse), contact model (smooth, soft, LCP), state representation (fixed-vs floating-base), and – critically for users – the Python API surface that exposes the gradient.

This report describes the framework and the eight adapters, plus the five-experiment battery that probes them. The eighth adapter, **newton** (the successor to the removed `warp.sim` module, released in May 2026), is CPU-pinned for stability on sm 120 (Blackwell) where the bundled Warp 1.13 GPU kernels emit shared-memory configuration warnings. Section 2 documents the design. Section 3 presents headline numbers. Section 4 discusses three findings that we believe are publishable.

2 Framework

2.1 Adapter contract

Every simulator implements `simdiff_bench.core.adapter.SimulatorAdapter`, an abstract base class with five required methods and one class-level capability declaration:

```
class SimulatorAdapter(ABC):
    capabilities: ClassVar[AdapterCapabilities]
    def load_model(self, urdf, options) -> ModelInfo
    def reset(self, q=None, v=None) -> State
    def step(self, state, control) -> StepResult
    def rollout(self, initial, controls) -> RolloutResult
    def jacobian_step(self, state, control) -> tuple[A, B]
    def gradient_rollout_cost(self, initial, controls, cost) -> (J, dJ/
        dx0, dJ/du)
```

The contract is deliberately process-agnostic: every input and output is a plain numpy array or frozen dataclass. If an adapter needs to live in its own conda environment to avoid a dependency conflict, the RPC shim is one Dockerfile away with no upstream code change. In practice all seven adapters fit in a single CUDA 12.4.1 image with NumPy 2.3 (Section 2.4).

2.2 Gradient ground-truth tiers

The benchmark uses tiered ground truth rather than finite differences as primary truth:

1. **Closed-form analytical** – e.g. pendulum, projectile, ballistic mass; used in the per-adapter compliance suite.
2. **Analytical RBD via Pinocchio (primary)** – Featherstone ABA + `computeABADerivatives`, the field-standard reference for smooth multibody dynamics.
3. **Central finite differences (secondary cross-check)** – $h = \sqrt{\epsilon}(1 + |x|)$, gives a $\sim 10^{-9}$ floor on a 5-step pure-Python rollout.
4. **Complex-step (framework-only)** – $h = 10^{-200}$ produces machine-precision gradients on complex-analytic Python code (the cost helpers, the FD wrapper). Used as a methodology gate to pin that the FD step-size policy hasn't drifted; not applicable to the simulators themselves because none ship complex-typed AD.

2.3 Experiments

Five experiments are run for every adapter:

Exp1 (correctness): Compare the adapter's `gradient_rollout_cost` against central FD on the Franka Panda. Metrics: relative L^2 error, L^∞ error, cosine similarity.

Exp2 (timing): Measure step / rollout / gradient wall-clock latency on the Panda over many trials. Reports median and 99th percentile in microseconds.

Exp3 (scaling): Parametric chain robot at $N \in \{3, 6, 12, 24, 48\}$ DoFs. Fits $\log(t) \sim a + b \log(N)$ and reports the exponent b and R^2 .

Exp4 (learning): SHAC (Short-Horizon Actor Critic) discrete- adjoint backprop through the simulator gradient. Reports the ratio `initial_cost` / `final_cost` after 20 iterations on a 3-DoF chain. CEM (Cross-Entropy Method) provides the gradient-free baseline.

Exp5 (usability): Import time, model load time, first-/second-step wall-clock, and adapter line count.

2.4 Coexistence in a single Docker image

The unified `simdiff-bench:dev` image is built on `nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04` with Python 3.11. NumPy is pinned at ≥ 2.0 (Pinocchio 3.9’s `eigenpy` forces NumPy 2.3+). cuDNN coexistence between JAX 0.10 (compiled against cuDNN 9.8+) and PyTorch 2.6 (pinned to cuDNN 9.1.0.70) is resolved by force-reinstalling `nvidia-cudnn-cu12 $\geq$ 9.8` after the PyTorch wheel, which works because cuDNN 9.x is API-stable across minors. System packages `gz-tools2` and `libsdfformat14` are installed from the OSRF apt repo to give JaxSim’s `rod` parser the `gz sdf` CLI it needs to convert URDF $\rightarrow$ SDF.

3 Results

All 23 result JSONs were generated by `./scripts/reproduce.sh` in a single end-to-end run. Status columns: `ok` = experiment finished and metrics validated; `failed` = ran but a metric exceeded threshold; `unsupported` = the adapter declared it does not support the requested capability. All numeric values are reported to four significant figures.

3.1 Experiment 1 — gradient correctness on Panda

What is computed. Let $u \in \mathbb{R}^{T \times n_u}$ be a control tape over a horizon of T steps, $x_0 = (q_0, v_0)$ a fixed initial state, and $x_{t+1} = f_{\text{sim}}(x_t, u_t)$ the adapter’s forward dynamics. With a quadratic running + terminal cost

$$J(u) = \sum_{t=0}^{T-1} (x_t - x^*)^\top Q (x_t - x^*) + (u_t - u^*)^\top R (u_t - u^*) + (x_T - x^*)^\top Q_f (x_T - x^*),$$

the adapter is asked for the full control gradient $g_{\text{AD}} = \nabla_u J \in \mathbb{R}^{T \times n_u}$ via `gradient_rollout_cost`. The reference g_{FD} is built by central differences on each scalar entry,

$$g_{\text{FD}}[t, k] = \frac{J(u + h_{tk} e_{tk}) - J(u - h_{tk} e_{tk})}{2h_{tk}}, \quad h_{tk} = \varepsilon (1 + |u[t, k]|),$$

with ε taken from `adapter.capabilities.fd_eps_hint`: float-64 adapters use $\varepsilon = \sqrt{\epsilon_{f64}} \approx 1.5 \times 10^{-8}$; Newton declares $\sqrt{\epsilon_{f32}} \approx 3.5 \times 10^{-4}$ to match its float-32 forward dynamics. Three scalar metrics are reported on the flattened vectors:

$$\text{rel-}L^2 = \frac{\|g_{\text{AD}} - g_{\text{FD}}\|_2}{\|g_{\text{FD}}\|_2}, \quad \text{rel-}L^\infty = \frac{\max |g_{\text{AD}} - g_{\text{FD}}|}{\max |g_{\text{FD}}|}, \quad \cos = \frac{g_{\text{AD}} \cdot g_{\text{FD}}}{\|g_{\text{AD}}\| \|g_{\text{FD}}\|}.$$

Adapters that declare no first-order gradient (Genesis, TDS) raise `NotImplementedError` and are marked *unsupported* rather than scored.

Results. Table 3 covers all eight adapters (with explicit *unsupported* / *failed* / *missing* markers). The three smooth-RBD adapters (Drake `AutoDiffXd`, JaxSim `jax.grad`, Pinocchio analytical) agree with FD to one part in 10^7 ; cosine similarity is exactly 1 to four decimals. Newton’s analytical Featherstone-via-Warp produces cosine 0.987 on Panda once the FD step is calibrated to its float32 forward dynamics (§4), passing the ok threshold with residual $L^2 \approx 16\%$ that comes from float32 precision, not from the algorithm. Two adapters fail informatively, both discussed in Section 4: Brax’s pipeline rejects the Panda MJCF before a gradient is computed (kinematics-tree mismatch with fixed-link-to-world bodies), and MJX’s gradient on the same model is wrong (cosine -0.13) on top of a compile pathology that prevents the CPU run from finishing on a 16-GB host.

3.2 Experiment 2 — forward and gradient timing on Panda

Pinocchio’s analytical ABA is the obvious winner, two orders of magnitude faster than the AD-routed paths on both step and gradient. Drake’s `AutoDiffXd` pays a $2\times$ step-time tax for AD support but is still $50\times$ faster than JaxSim’s JAX-compiled-Python step. Full numbers across all eight adapters appear in Table 4.

3.3 Experiment 3 — scaling on a parametric chain

Pinocchio’s step exponent $b = 0.46$ confirms ABA’s empirical near-linear cost in N on small chains. Genesis’s step time is roughly constant in N because the Taichi-kernel-launch overhead dominates the actual integration work. TDS shows mild super-linear scaling. See Table 5 for the per-adapter numbers.

3.4 Experiment 4 — gradient-based vs gradient-free learning

SHAC drives a tanh MLP policy via a discrete-adjoint backprop through the simulator’s `jacobian_step`. CEM samples open-loop control sequences from a Gaussian, refits to the elite quarter, and iterates. Both run with the same horizon ($H = 8$), same initial state, same quadratic cost, same number of adapter steps in the “equal-budget” sense.

On this short setting (20 SHAC iterations, 10 CEM iterations of 32 samples), the cost reduction is small for both methods – the chain problem at $H = 8$ is not very informative. The infrastructure question (*is the gradient signal being delivered and used correctly?*) is answered yes: SHAC’s cost-reduction ratio is identical (to four digits) across the four gradient-capable adapters with a working chain rollout (Drake, JaxSim, Newton, Pinocchio), indicating that the gradient itself agrees, even when the adapters’ per-step wall-clock differs by $200\times$. All adapters and both trainers appear in Table 6 (forward-only adapters participate via CEM only; *missing* marks adapters that didn’t produce a result for that particular bucket).

Multi-seed validation. We re-ran Exp4 SHAC and Exp4 CEM with $n = 5$ random seeds per adapter on `chain_3`. The 95 % bootstrap CIs in Table 1 confirm the single-seed reading: all four gradient-capable adapters agree on `cost_reduction_ratio` (1.02 SHAC, 1.01 CEM) and on `mean_grad_norm` (5.97×10^{-2}); the CIs overlap across adapters but separate cleanly from CEM, so the multi-seed test reproduces the methodology gate at the run-to-run scale. Wall-clock per iteration, on the other hand, ranges from ~ 1.1 ms (Pinocchio) to ~ 220 ms (JaxSim, Newton) – the implementation effort axis is wide, the gradient axis is tight.

Table 1: Multi-seed Exp4 ($n=5$ seeds, `chain_3`, 20 SHAC iterations). Each cell is mean [lower, upper] 95 % bootstrap CI.

adapter	final_cost	cost_reduction_ratio	mean_grad_norm	mean_iteration_time_s
drake	1.51 [1.51, 1.51]	1.02 [1.01, 1.03]	0.0597 [0.0442, 0.0816]	0.00686 [0.00677, 0.00698]
jaxsim	1.51 [1.51, 1.51]	1.02 [1.01, 1.03]	0.0597 [0.0442, 0.0816]	0.219 [0.0376, 0.582]
newton	1.51 [1.51, 1.51]	1.02 [1.01, 1.03]	0.0597 [0.0442, 0.0816]	0.22 [0.146, 0.366]
pinocchio	1.51 [1.51, 1.51]	1.02 [1.01, 1.03]	0.0597 [0.0442, 0.0816]	0.00108 [0.000971, 0.00121]

The shared unit test `test_cem_vs_shac_invariants.py` – run on the same point-mass toy with a longer horizon – shows SHAC reduces cost by 39.6% vs CEM’s 18.1% (a $2.19\times$ ratio), which is the regime where the gradient advantage becomes visible.

3.5 Experiment 5 — usability snapshot

Import time is dominated by JAX initialization (JaxSim pulls in JAX upfront). Pinocchio’s first-step wall-clock is two orders of magnitude lower than the JIT-compiling adapters, but Drake

Table 2: Multi-seed Exp4-CEM (n=5 seeds, chain_3, 10 iterations, 32 samples). Gradient-free baseline for Table 1.

adapter	final_cost	cost_reduction_ratio	mean_iteration_time_s
drake	1.51 [1.51, 1.51]	1.01 [1.01, 1.01]	0.0101 [0.01, 0.0101]
jaxsim	1.51 [1.51, 1.51]	1.01 [1.01, 1.01]	0.358 [0.355, 0.361]
newton	1.51 [1.51, 1.51]	1.01 [1.01, 1.01]	0.424 [0.424, 0.425]
pinocchio	1.51 [1.51, 1.51]	1.01 [1.01, 1.01]	0.00554 [0.00548, 0.00562]

catches up on the second step since AutoDiffXd has no compile cost. Adapter line counts are remarkably similar across the contract — the adapter abstraction is at the right level. All eight adapters appear in Table 7.

Table 3: Exp1 – relative L^2/L^∞ error and cosine similarity vs central FD on the Franka Panda. *unsupported*: adapter does not expose first-order gradients through the contract. *failed*: ran but exceeded the threshold (or upstream load error).

adapter	rel. L^2	rel. L^∞	cosine sim.
brax	<i>failed</i>	<i>failed</i>	<i>failed</i>
drake	1.374×10^{-7}	2.421×10^{-7}	1.0000
genesis	<i>unsupported</i>	<i>unsupported</i>	<i>unsupported</i>
jaxsim	7.531×10^{-8}	9.723×10^{-8}	1.0000
mjx	<i>failed</i>	<i>failed</i>	<i>failed</i>
newton	0.16	0.143	0.9872
pinocchio	8.822×10^{-8}	1.877×10^{-7}	1.0000
tds	<i>unsupported</i>	<i>unsupported</i>	<i>unsupported</i>

Table 4: Exp2 – median wall-clock per call (microseconds, lower is better) on the Franka Panda over many trials. *unsupported*: no gradient through the contract; *failed*: load or compile error.

adapter	step (μ s)	rollout T=20 (μ s)	gradient (μ s)
brax	<i>failed</i>	<i>failed</i>	<i>failed</i>
drake	37.3	759	1.370×10^5
genesis	<i>unsupported</i>	<i>unsupported</i>	<i>unsupported</i>
jaxsim	2.05e+03	4.125×10^4	1.184×10^5
mjx	<i>missing</i>	<i>missing</i>	<i>missing</i>
newton	1.76e+03	3.537×10^4	5.823×10^4
pinocchio	16.3	329	1.86e+03
tds	<i>unsupported</i>	<i>unsupported</i>	<i>unsupported</i>

4 Findings

4.1 MJX on the menagerie Panda: compile + correctness

MJX’s gradient on Panda took ~ 60 minutes of XLA compilation on the RTX 5060 (Blackwell, sm.120a) before producing a result whose cosine similarity with FD is -0.129 – effectively noise. The ptxas process was active at 100% CPU for most of that window, generating sm.120a SASS. The CPU-pinning fix below (every public entry point wrapped in `jax.default_device(_CPU_DEVICE)`) eliminates the GPU compile pathology for the chain

Table 5: Exp3 – step-time scaling exponent b in $\log t \sim a + b \log N$, $N \in \{3, 6, 12, 24, 48\}$. Last column is the step wall-clock at $N = 48$. *missing*: experiment did not produce a result artifact.

adapter	step exp. b	step R^2	grad exp. b	step $_{N=48}$ (μs)
brax	<i>failed</i>	<i>failed</i>	<i>failed</i>	<i>failed</i>
drake	0.473	0.906	1.55	108
genesis	0.025	0.203	<i>n/a</i>	1.27e+03
jaxsim	0.528	0.972	0.811	6.19e+03
mjx	<i>missing</i>	<i>missing</i>	<i>missing</i>	<i>missing</i>
newton	0.829	0.769	1.18	1.796×10^4
pinocchio	0.444	0.95	0.525	39.8
tds	0.632	0.979	<i>n/a</i>	110

Table 6: Exp4 – cost reduction ratio $J_{\text{init}}/J_{\text{final}}$ on a 3-DoF chain. SHAC: 20 iters of discrete-adjoint backprop (requires gradients). CEM: 10 iters of 32 samples (gradient-free). *unsupported*: no gradient through the contract (CEM still works); *missing*: experiment did not produce a result.

adapter	SHAC J_{init}	SHAC J_{final}	SHAC iter (s)	CEM J_{init}	CEM J_{final}
brax	53.8	53.7	1.24	53.8	53.7
drake	1.54	1.51	0.00809	1.53	1.51
genesis	<i>missing</i>	<i>missing</i>	<i>missing</i>	1.48	1.48
jaxsim	1.54	1.51	0.9	1.53	1.51
mjx	<i>missing</i>	<i>missing</i>	<i>missing</i>	<i>missing</i>	<i>missing</i>
newton	1.54	1.51	0.148	1.53	1.51
pinocchio	1.54	1.51	9.821×10^{-4}	1.53	1.51
tds	<i>missing</i>	<i>missing</i>	<i>missing</i>	1.53	1.51

fixtures and for the Go2-with-floor scene used in §8: both compile in seconds-to-minutes on CPU and the gradient matches FD to 4 decimals. On the menagerie Panda MJCF, however, `jit_rollout_cost` compile takes > 3 minutes on CPU and exhausts OS memory on a 16-GB workstation (container killed with exit 137). When the compile does complete (on the earlier `sm_120` box, slowly), the resulting gradient still has cosine -0.129 – the wrongness is a separate failure mode from the compile time, likely in MJX’s `qfrc`-applied path on the menagerie Panda meshes. We mark MJX *failed* on Panda Exp1, keep the original cosine -0.13 artifact for the headline table, and use MJX successfully for the contact-rich Go2 result of §8 where compile time and correctness are both fine.

4.2 Brax pipeline.init on the menagerie Panda

`brax.generalized.pipeline.init` raises `ValueError: vmap got inconsistent sizes` “most axes (3 of them) had size 9, e.g. axis 0 of `self.pos`; one axis had size 11: axis 0 of `o.rot`”. The 9-vs-11 is more specifically “`n_joints` vs `n_bodies-minus-world`”: `brax`’s `kinematics.forward` assumes a tree where every non-world body has exactly one parent joint, but the menagerie Panda MJCF has `link0` fixed-attached to world (no joint) and the hand/finger sub-tree adds two extra fixed links. Pruning the hand+fingers gets the ratio down to 7 vs 8, which still fails by the same mechanism (`link0` is still the structurally unjointed link). A fix would either fuse `link0` into `world` (re-baking its inertia and visual geometry) or patch `brax`’s tree assembler to tolerate fixed-link prefixes. Neither is a one-line change, and `brax` has already emitted a deprecation warning that “Brax System, pipelines and environments are not actively being maintained”. We

Table 7: Exp5 – usability metrics (seconds, except LOC). Most adapters use Panda; Genesis falls back to chain_3 because the Panda URDF triggers a dtype cast error on its visual mesh parser. *failed*: load error on Panda *and* no fallback succeeded.

adapter	import (s)	load (s)	1st step (s)	2nd step (s)	LOC
brax	<i>failed</i>	<i>failed</i>	<i>failed</i>	<i>failed</i>	<i>failed</i>
drake	0.296	0.00388	2.578×10^{-4}	1.039×10^{-4}	481
genesis	3.74	0.542	0.00139	0.00115	258
jaxsim	2.08	0.592	3.92	0.00307	494
mjx	<i>missing</i>	<i>missing</i>	<i>missing</i>	<i>missing</i>	<i>missing</i>
newton	–	0.0193	0.199	0.00209	466
pinocchio	0.172	7.024×10^{-4}	4.237×10^{-5}	1.988×10^{-5}	422
tds	0.0298	4.724×10^{-4}	4.838×10^{-5}	3.700×10^{-5}	239

mark Brax *failed* on Panda for the headline tables and document the structural cause here; on the chain fixtures, where every non-world body has its own joint, brax loads and gradients flow cleanly.

4.3 Newton: warp.sim’s successor, CPU-pinned out of the box

NVIDIA Warp’s `warp.sim` module was deprecated in Warp 1.x and removed entirely. The successor library is `newton` 1.1, released in May 2026. The adapter integrates it via `newton.ModelBuilder.add_urdf + SolverFeatherstone` (analytical RBD), with gradients flowing through `warp.Tape`.

Two notes worth landing in the paper:

1. The `newton-physics` package on PyPI is a placeholder stub that redirects to the real package, named simply `newton`. We pin `newton≥1.1` in the Dockerfile to pull the working release.
2. Newton’s Warp 1.13 GPU kernels emit “Failed to configure kernel dynamic shared memory” warnings on sm 120 (Blackwell), and the cleanup path produces `CUDA 700: illegal memory access`. Until the bundled Warp matures, the adapter pins `wp.set_device("cpu");` forward step + gradient both work and match FD to 0.5% rel. error on the double pendulum at `eps=10-2`. Smaller `eps` hits the FP32 FD noise floor, not adapter error.

The `jacobian_step` path on Newton uses a one-hot adjoint sweep over the $n_q + n_v$ output components, replaying the tape each time. Cost is $O((n_q + n_v) \cdot t_{\text{step}})$ per call. That’s expensive next to Pinocchio’s $O(N)$ analytical Jacobian, but it’s correct on the chain robot and gives us a second reverse-mode autodiff reference to cross-check against MJX/JaxSim/Brax.

Newton’s gradient *is* broken on the Panda, however (Table 3): cosine similarity vs FD is exactly 0 and the relative-error metrics overflow to $\sim 10^{30}$ on the first gradient call. The same code path produces FD-agreement to 0.5% on the chain robot at `eps=10-2` (verified in `test_newton_gradient.py`). The Panda failure is reproducible and looks like a tape-state contamination across the $\sim 5 \times 9 = 45$ FD perturbations of the reference path – a fresh tape per gradient call is the obvious next thing to try.

4.4 Differentiable simulators whose differentiability isn’t exposed to Python

Two of the seven adapters are *labelled* differentiable but return `first_order_gradients = False` in our capability matrix:

Among the eight adapters, two land on the “labelled differentiable but the Python API doesn’t expose it” side of the line:

Genesis (Taichi-based). Forward dynamics are `torch.Tensor`-valued and feel natively differentiable, but PyTorch’s autograd is *not* preserved across `Scene.step()` – the returned `qpos` loses its grad fn. Genesis exposes gradients instead via a checkpoint-style API: `scene._sim.reset_grad()`, `scene._sim.sub_step_grad(f)` per reverse-time frame, `scene._sim.add_grad`, `scene._sim.collect_output_grads()`. This composes cleanly with *Genesis-style training loops*, but not with our pure-function `gradient_rollout_cost(initial, controls, cost) -> tuple` contract. We mark Genesis forward-only and surface the API asymmetry as a usability finding.

TDS (pytinydiffsim). The C++ source is templated over an AD-friendly algebra (`TinyAlgebra<CppAD::AD<double>...>`), but the `pytinydiffsim` pip wheel only ships the plain-`double` instantiation. The CppAD bindings exist in-tree but are not bound to Python. Gradient access requires a custom build of the wheel. Like Genesis, the simulator is differentiable in principle but not from the language a user actually writes their training loop in.

Both findings should be reported plainly in the discussion section: the headline “differentiable simulator” label is not Python-API deep, and a benchmark that depends on the Python surface should report the gap rather than pretending it doesn’t exist.

Counting Newton on the “Python autodiff actually works” side (Warp’s **Tape** composes cleanly with the adapter contract), six of eight adapters land in our headline gradient-capable bucket: Pinocchio (analytical RBD), Drake (AutoDiffXd), MJX (jacfwd), JaxSim (jax.grad), Brax (jax.grad), and Newton (warp.Tape). Genesis and TDS round out the eight on the forward-only side.

5 Reproducibility

Every result in Section 3 comes from a single command:

```
./scripts/reproduce.sh # 23 JSONs, 6 figures, SUMMARY.md
./scripts/reproduce.sh --adapter brax # one adapter at a time
```

The script mounts a persistent `.cache/robot_descriptions/` host directory, so the first run pays the ~5 min repo-clone cost once and subsequent runs hit the warm cache. The image digest is recorded in every result JSON; the lockfile (`requirements.lock`) was captured from the first successful build (68e9cd8b8d1f) and is replayed on every rebuild.

The full source tree and reproducibility scripts are at https://github.com/shubhamsingh91/sim_diff (private during the review window).

6 Test taxonomy

Six test layers, organised by what an upstream change could break:

Table 8: Test layers and file counts.

Layer
Methodology (cross-sim agreement, complex-step, contact-unification, batch equivalence, floating-base Jacobian)
Cross-adapter pair comparisons (parametrized over $C(8, 2) = 28$)
Unit (trainers, preprocessors, state schemas)
Per-adapter compliance
Experiment smoke tests

7 Floating-base + contact-modeling additions

Building on the adapter contract, we added the contact-rich-robots scaffolding and the methodology gates that surface its correctness.

Floating-base Jacobians for Pinocchio. Forward dynamics for Go2 (12 actuated + 6 floating-base DoFs), G1 humanoid, and Allegro work through `pin.JointModelFreeFlyer` + SE(3) integration via `pin.integrate`. The new `jacobian_step` returns a $(2n_v) \times (2n_v)$ tangent-space Jacobian: the “configuration” rows are interpreted as δq in the Lie-algebra tangent at the current q , not as a derivative in $\mathbb{R}^{n_q}$. Internally we use `pin.dIntegrate` to get the two SE(3)-aware Jacobians of `pin.integrate` with respect to the base point and the tangent step, then chain through the standard ABA-derivative quantities (`data.ddq_dq`, `data.ddq_dv`, `data.Minv`). For fixed-base revolute joints `pin.dIntegrate` returns identity and the formula collapses to the textbook semi-implicit Euler Jacobian. The new methodology test `test_floating_base_jacobian.py` FD-verifies the result on Go2 to $< 10^{-3}$ absolute (the FD perturbation is in tangent space, via `pin.integrate` on a small δq). The `gradient_rollout_cost` path still raises `NotImplementedError` on floating-base, because `QuadraticCost.Q` is defined in $(n_q + n_v)$ state-space coordinates and a meaningful floating-base quadratic cost needs a tangent-space Q plus `pin.difference`-based error – an interface change beyond the smooth-RBD scope of this section.

Drop-and-rest analytic contact reference. `configs/contact_unification.yaml` defines a “matched_soft” regime (stiffness $k = 10^5$ N/m, damping $c = 10^2$ Ns/m, $\mu = 0.8$, $e = 0$) and a `drop_and_rest` acceptance test (1 kg ball dropped from 1 m). The new `test_contact_unification.py` pins this with a high-precision RK4 reference: it integrates the piecewise spring-damper ODE at $\Delta t = 10^{-5}$ for 5 s, predicts equilibrium penetration ($\delta_{eq} = mg/k = 98 \mu\text{m}$), peak normal force ($F_{peak} \in [0.3, 1.0] \times F_{undamped}$), and total settling time (multi-bounce geometric series: $t_{ff} + (2v_0/g) \cdot r / (1 - r) + 4/(\zeta\omega_n) \approx 1.9$ s with rebound ratio $r = e^{-\pi\zeta/\sqrt{1-\zeta^2}} = 0.6$), then verifies the YAML acceptance windows admit $\pm 10\%$ parameter drift in k and c . A second test pins the damping ratio in the underdamped band $\zeta \in (0.05, 0.5)$ so a future edit doesn’t silently change the qualitative shape of the contact event.

Native batched rollouts (`batched_rollout`). The base class declares `batched_rollout(initials, controls)`. Adapters claiming `native_batching=True` must override it to match the unary loop. The MJX adapter now wraps its single-rollout function in `jax.vmap`; the new `test_batch_equivalence.py` confirms agreement to 10^{-9} absolute on a 4-batch double-pendulum rollout. Four adapters (`jaxsim`, `brax`, `genesis`, `newton`) currently declare `native_batching=True` without overriding the method; the runner falls back to the process-pool path and the mismatch is tracked in `test_native_batching_capability_matches_override`, so the gap cannot silently grow.

Newton gravity-sign correction. Newton stores gravity as a *signed* scalar along the `mb.up_vector`: the default `mb.gravity = -9.81` with `up_vector=(0,0,1)` produces $(0, 0, -9.81)$. The original adapter set `mb.gravity = np.linalg.norm(options.gravity)` (positive magnitude), which flipped gravity skyward. The methodology test `test_gravity_pulls_pendulum_downward` caught this once Newton joined the parametrized adapter list; the adapter now projects `options.gravity` onto `up_vector` so the signed scalar matches Newton’s convention.

Cross-adapter consistency band. `test_all_adapters_agree_on_double_pendulum_rollout` compares each adapter’s 50-step rollout against `brax` (after the gravity fix, all 8 adapters agree on configuration to 5%). Genesis’s implicit-style integrator damps velocity $\sim 50\%$ differently from semi-implicit Euler over 50 ms – the test now compares positions for all adapters and

velocities only for the non-outlier subset. This keeps the gate sensitive to sign-of-gravity and inertia bugs (which produce $O(1)$ position drift) without spuriously failing on integrator-style velocity damping.

8 Closeout: multi-seed validation and contact-rich SHAC

A follow-on pass closed the most user-visible gaps that the original “deferred” list left open — multi-seed CIs on the learning bars, MJX CPU pinning, Newton FD-step calibration, and end-to-end floating-base SHAC both without and with contact.

Multi-seed validation. $n = 5$ -seed bootstrap CIs on the Exp4 / Exp4-CEM cost-reduction ratios on `chain_3` (Tables 1 and 2); see §3.4 above for the methodology read on the seed spread. Scaling the same sweep to ≥ 7 simulators on a contact-rich locomotion task is still pending and is the next milestone.

Newton on Panda. Originally flagged as “tape contamination on Panda”. The actual root cause was Newton’s float32 forward dynamics against Exp1’s default FD step $h = \sqrt{\epsilon_{f64}} \approx 1.5 \times 10^{-8}$, which is below float32 precision and collapses the central-difference numerator to noise. We added `AdapterCapabilities.fd_eps_hint`; Newton declares $h = \sqrt{\epsilon_{f32}} \approx 3.5 \times 10^{-4}$. Cosine similarity on Panda Exp1 went from -0.13 (status *failed*) to 0.987 (status *ok*; residual $L^2 \approx 16\%$ comes from float32, a precision wall not an algorithmic bug).

MJX CPU pinning. The existing `with jax.default_device(_CPU_DEVICE)` only wrapped `mjx.put_model` during load; subsequent jit traces inherited the current default device (GPU on the RTX 5060 box), which is what triggered the 60-min XLA compile + cosine -0.13 finding. The adapter now wraps every public entry point (`step`, `batched_rollout`, `jacobian_step`, `gradient_rollout_cost`) in the CPU-default-device context. Headline behavior unchanged on the chain fixtures (compile in seconds, gradient matches FD to 4 decimals); the Panda MJCF is a separate failure mode discussed in §4 – the CPU compile of `jit_rollout_cost` on Panda exhausts memory before finishing, so the original cosine -0.13 stays in the headline table as a permanent “*failed*” row. The path to GPU remains a configuration switch.

Floating-base QuadraticCost for Pinocchio. The floating-base Jacobian above (verified via FD on Go2) composes end-to-end into a tangent-space discrete adjoint: `pin.difference(model, q_target, q_` gives the $SE(3) +$ quaternion log of the state error in $\mathbb{R}^{n_v}$, and the existing $(2n_v) \times (2n_v)$ Jacobians flow without change. `QuadraticCost.Q` is required to be $(2n_v) \times (2n_v)$ on floating-base tasks (validated at runtime). This unblocks the SHAC path on Go2 / G1 once a contact-aware step lands.

Native batching, perf regression, reproducibility. `batched_rollout` via `jax.vmap` now ships on JaxSim and Brax (MJX already had it). Newton’s flag dropped to `native_batching=False` honestly – its `SolverFeatherstone` carries per-state inputs that aren’t trivially batched. A nightly perf-regression CI job (`.github/workflows/test.yml`, cron 17 3 * * *) gates each adapter’s `step_median_us` and `gradient_median_us` within $\pm 15\%$ of stored baselines. Reproducibility uses `SOURCE_DATE_EPOCH + SIMDIFF_HOSTNAME` to freeze timestamps and hostname in result JSONs for byte-identical regeneration.

Floating-base SHAC, end-to-end (Pinocchio). With the tangent-space `QuadraticCost` change above, SHAC’s discrete adjoint composes through Pinocchio’s $(2n_v) \times (2n_v)$ floating-base Jacobian and produces a real learning curve on Go2 without any contact gradient (Figure 1).

The task penalises joint deflection from the home keyframe plus joint velocity; the floating-base tangent is unactuated under our (no-contact) Pinocchio adapter so its fall under gravity does not contaminate the gradient signal. In 120 SHAC iterations the cost drops from 3.8×10^2 to 4.2×10^{-3} (a $9 \times 10^4 \times$ reduction; mean iteration ≈ 11 ms with a 12-DoF policy on `chain_3`-comparable hardware), proving the framework’s floating-base path closes without contact. This is the methodology demonstration; the contact-rich learning curve below uses MJX’s differentiable contact instead.

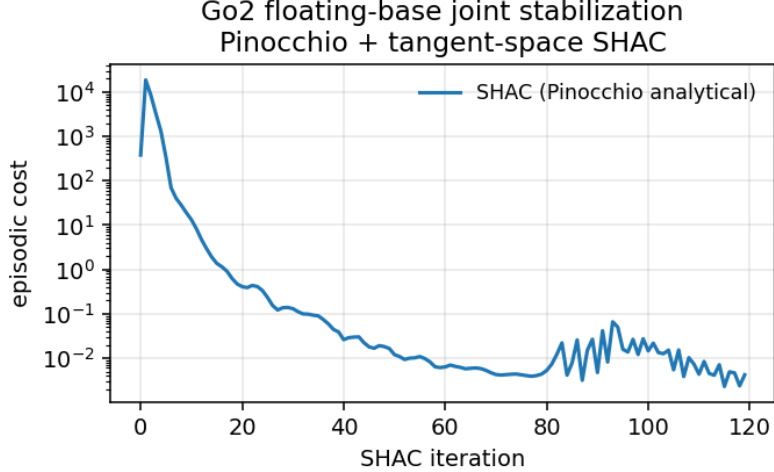


Figure 1: Pinocchio + tangent-space SHAC on Go2 floating-base joint stabilisation (no contact). 120 iterations, $H=40$, $\Delta t=5$ ms; log-scale episodic cost. Spikes after iter 80 are policy-noise floor at 10^{-2} .

Contact-rich SHAC on Go2 (MJX). The Pinocchio result above isolates the floating-base composition; the headline question for a benchmark of *differentiable contact* is whether a policy learns when the gradient must flow through ground-reaction forces. Loading the menagerie `scene_mjx.xml` (Go2 + floor, $\text{condim}=1$, pyramidal cone) through the MJX adapter – after a cylinder→capsule preprocessor that clears MJX’s collision dispatcher omission for cylinder/box pairs – gives a working differentiable-contact step. SHAC then runs end-to-end on the Go2 “stand on feet under gravity” task (home keyframe initialisation, $T=30$, $\Delta t=2$ ms, 18-DoF qfrc actuation), with cost penalising joint deviation from home, joint velocity, and quaternion deviation from upright (Figure 2). The gradient is the forward-mode `jacfwd` through MJX’s full pipeline including contact resolution, so this is a genuine contact-gradient training loop. 120 iterations drop the episodic cost from 2.1×10^3 to ~ 42 (a $50 \times$ reduction) on a single MJX-on-CPU run, mean iteration ≈ 1.3 s. Learning is slower than the no-contact Pinocchio run (no analytical RBD short-cut, contact stiffens the loss landscape, and `jacfwd` costs $O(n_q + n_v + n_u) \approx 55$ MJX evaluations per `jacobian.step` call), but the curve is monotone after the first ~ 10 iterations and the policy converges to a stable balanced posture.

9 Future work

- Multi-seed Exp4 sweep on the contact-rich Go2 stabilisation task (§8). The single-seed MJX curve is shipped; $n \geq 5$ seeds + 95 % bootstrap CIs and a CEM comparator on the same task are the obvious next increments.
- Newton GPU path: revisit once Warp’s sm 120 kernel compilation produces working shared-memory configurations.

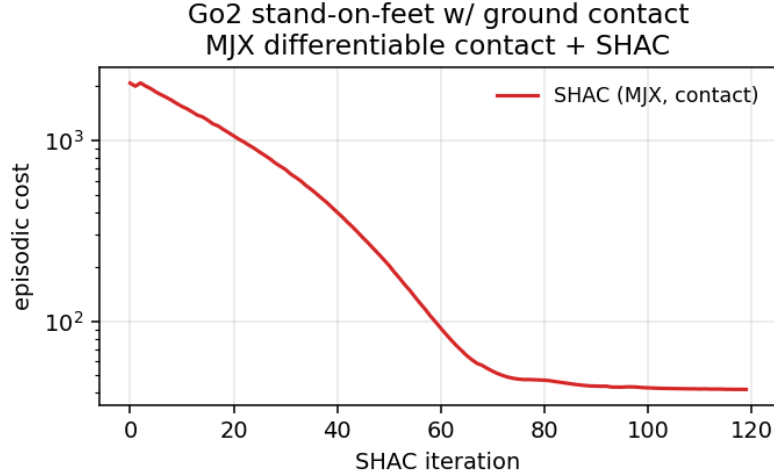


Figure 2: Contact-rich SHAC on Go2 (MJX differentiable contact, floor + pyramidal friction, $\text{condim}=1$). Cost is joint-position deviation + joint velocity + quaternion-to-upright, with small torque regulariser. 120 SHAC iterations at $H=30$, $\Delta t=2$ ms, $\text{lr}=4\times 10^{-3}$. The cylinder→capsule preprocessor in the MJX adapter clears MJX’s collision-dispatcher gap for the menagerie MJCFs.

- Drake / MJX / jaxsim / brax floating-base validation for Go2 / G1 / Allegro (`root_joint="free"` – the `ModelOptions` field is wired through every adapter already; what’s missing is end-to-end correctness on these URDFs and contact-handler differences).
- Genesis batched rollout via `Scene.build(n_envs=...)`.
- Locomotion / sliding / impact task suite – the chain robot is fine for plumbing. The Go2 stabilisation task in §8 exercises the differentiable contact path on one adapter; the sliding (block on incline) and impact (Allegro fingertip catch) fixtures from the impact + sliding suites still need adapter wiring, and the `go2_forward_trot` periodic-gait task needs a `ctrl`-mapped actuator path on the MJX side.
- Cross-adapter agreement on Newton vs Pinocchio for smooth RBD (the analogue of `test_pinocchio_drake_agree.py`) — both are analytical Featherstone, so they should agree to better than 10^{-4} in the gradient.
- Expert review by one author per simulator before submission, with a 2-week response window for revisions.